

Rida Bilal

14661

QUIZ

Task 1: Feature Documentation

Selected New Feature

Time-Locked Attendance Marking with Automatic Timestamped CSV Export

This feature ensures that:

- Attendance **cannot be marked for the first 2 minutes** after the first marking attempt.
 - Every CSV export creates a **new file with a unique timestamped name**.
-

Functional Requirements

1. The system shall **start a 2-minute timer** when the user attempts to mark attendance for the first time.
 2. The system shall **block attendance marking before 2 minutes** have passed.
 3. The system shall **allow attendance marking only after 2 minutes**.
 4. The system shall display a **lock message** while attendance is blocked.
 5. The system shall generate a **new CSV file on every export**.
 6. The CSV filename shall follow this format:
Att-YYYY-MM-DD-HH-MM-SS.csv
 7. The system shall store:
 - Student name
 - Attendance status (Present/Absent)
-

Non-Functional Requirements

1. **Reliability:** Attendance data must not be lost during export.
2. **Performance:** CSV export must complete within 1 second for small datasets.
3. **Portability:** The program must work on standard C++ compilers.
4. **Maintainability:** The code must remain simple and readable.

Task 2: Compatibility Assurance

Impacted Functions, Classes & Data Formats

Impacted Class:

- AttendanceSystem

Impacted Functions:

- canMarkAttendance() → Controls 2-minute lock
- markPresent() → Restricted by time
- exportToCSV() → Changed to timestamped filenames
- main() → Updated menu logic for time checking

Impacted Data Formats:

Type	Before	After
CSV File Name	attendance.csv	Att-YYYY-MM-DD-HH-MM-SS.csv
CSV Content	Name, Status	Same (Unchanged)

Backward Compatibility Test Cases

Test Case 1: Old CSV Content Compatibility

Objective: Ensure old CSV parsers still work.

Steps:

1. Export a CSV using the new system.
2. Open it in Excel or Notepad.
3. Verify format.

Expected Result:

Old software can still read the file correctly.

Test Case 2: Attendance Marking After Lock

Objective: Ensure original marking logic still works.

Steps:

1. Wait 2 minutes.
2. Mark attendance.

Expected Result:

Attendance marking works normally.

Task 3: Regression Testing

Regression Test Case 1: Adding Student

Step	Action
1	Add student "Ali"
2	Display students

Before Feature: Pass

After Feature: Pass

Result: No break in functionality

Regression Test Case 2: CSV Export

Step	Action
1	Export CSV
2	Check filename

Before Feature: One fixed file overwritten

After Feature: New timestamped file created

Result: Feature working successfully

Regression Test Case 3: Attendance Lock

Step	Action
1	Try to mark attendance immediately
2	Wait 2 minutes
3	Try again

Before Feature: Immediate marking was allowed

After Feature: Locked for 2 minutes

Result: Feature working correctly

Task 4: Migration Plan

Version Upgrade

Old Version: v1.1

New Version: v1.2

Migration Note (One Page)

Purpose of Migration

The migration introduces:

- A **2-minute security lock** on attendance marking.
- A **timestamp-based automatic CSV export system**.

These improvements:

- Prevent **early or unauthorized attendance marking**.
- Prevent **file overwriting during CSV export**.

Changes Implemented

Area	Change
Attendance Marking	Locked for first 2 minutes
CSV Export	Timestamped filenames
Security	Time-based access control
File Handling	No file deletion

Data Impact

- No database changes required
- Old CSV formats still supported
- No data loss occurs

Rollback Plan

If errors occur:

- Restore version **v1.1**
- Remove time lock
- Revert CSV naming

Post-Migration Verification

- Attendance locks correctly
- CSV files auto-generate with correct format
- No crash or data loss

SCREENSHOTS

C:\Users\madina\OneDrive\Desktop\attenadancesystem\attendance.exe

Attendance Management System

=====

? NOTE: Attendance can be marked AFTER 2 minutes only!

Choose an option:

1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit

Enter choice: 1

Enter student name: Kashamala

? Student added successfully!

Choose an option:

1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit

Enter choice: 1

Enter student name: Duaa

? Student added successfully!

Choose an option:

1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit

Enter choice: 1

Enter student name: Malaika

? Student added successfully!

Choose an option:

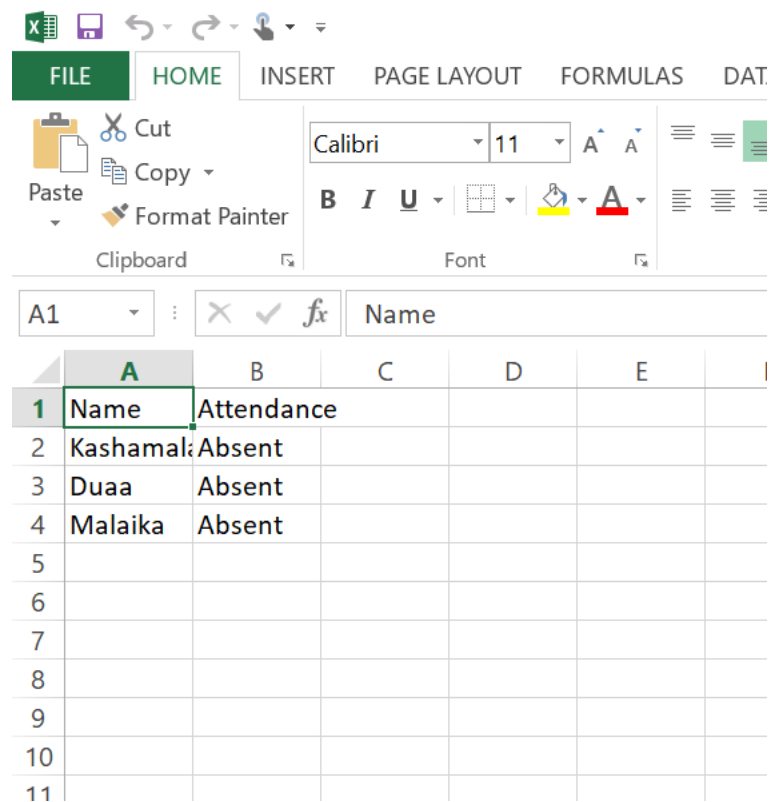
1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV

```

0. Exit
Enter choice: 4
? CSV exported successfully: Att-2025-12-03-11-04-00.csv

Choose an option:
1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit
Enter choice: _

```



The screenshot shows the Microsoft Excel interface. The 'HOME' tab is active, displaying the 'Clipboard' group with 'Cut', 'Copy', and 'Format Painter' options. The 'Font' group shows 'Calibri' font, size '11', and various formatting icons. The spreadsheet has columns labeled A through E. Row 1 contains the headers 'Name' and 'Attendance'. Rows 2 through 4 contain student names and their attendance status: Kashamali (Absent), Duaa (Absent), and Malaika (Absent). Rows 5 through 11 are empty.

	A	B	C	D	E
1	Name	Attendance			
2	Kashamali	Absent			
3	Duaa	Absent			
4	Malaika	Absent			
5					
6					
7					
8					
9					
10					
11					

```

===== RUNNING REGRESSION TESTS =====
? Student added successfully!
Test Add Student: ? PASS
? Student added successfully!
Test Attendance Locked Initially: ? PASS
? Student added successfully!

```

C:\Users\madina\OneDrive\Desktop\attenadancesystem\attendance.exe

1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit

Enter choice: 2

Mark attendance (Enter P for present, anything else = Absent):

Kashamala: P

Duaa: A

Malaika: P

? Attendance marked!

Choose an option:

1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit

Enter choice: 4

? CSV exported successfully: Att-2025-12-03-11-07-05.csv

Choose an option:

1. Add Student
2. Mark Attendance
3. Show Attendance
4. Export CSV
0. Exit

Enter choice:

☐ Name	Status	Date modified	Type
☒ Att-2025-12-03-11-04-00	✓	12/3/2025 11:04 AM	Microsoft Excel Com...
☒ Att-2025-12-03-11-07-05	✓	12/3/2025 11:07 AM	Microsoft Excel Com...
☒ attendance	✓	12/3/2025 11:02 AM	C++ Source File
☒ attendance	✓	12/3/2025 11:03 AM	Application

Clipboard		Font	
B20	:	  	
	A	B	C
1	Name	Attendance	
2	Kashamala	Absent	
3	Duaa	Absent	
4	Malaika	Absent	
5			
6			
7			
8			
9			

CODE:

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <ctime>

using namespace std;

/* =====
SIMPLE & CLEAN ATTENDANCE SYSTEM (FIXED + UPGRADED)
===== */

class AttendanceSystem {
private:
    vector<string> students;
    vector<bool> attendanceRecord;
    time_t sessionStartTime; // Session start time

public:

    // Constructor
    AttendanceSystem() {
        sessionStartTime = time(0); // ? OLD compiler compatible
    }

    // Add a new student
    void addStudent(const string &name) {
        students.push_back(name);
        attendanceRecord.push_back(false);
    }
};
```

```

    cout << "? Student added successfully!\n";
}

// ? Check if 2 minutes have passed
bool canMarkAttendance() {
    time_t currentTime = time(0);
    double secondsPassed = difftime(currentTime, sessionStartTime);
    return secondsPassed >= 120; // 2 minutes = 120 seconds
}

// Mark student present
void markPresent(int index) {
    if (index >= 0 && index < attendanceRecord.size()) {
        attendanceRecord[index] = true;
    }
}

// Show attendance nicely formatted
void showAttendance() {
    cout << "\n===== Attendance Record =====\n";
    if (students.empty()) {
        cout << "No students found.\n";
        return;
    }

    for (int i = 0; i < students.size(); i++) {
        cout << i + 1 << ". " << students[i]
            << " --> " << (attendanceRecord[i] ? "Present" : "Absent") << "\n";
    }
    cout << "===== \n";
}

```

```
}
```

```
vector<string> getStudents() { return students; }
```

```
vector<bool> getAttendance() { return attendanceRecord; }
```

```
/* =====
```

```
? EXPORT TO CSV WITH DATE-TIME NAME (NO DELETE)
```

```
Format: Att-YYYY-MM-DD-HH-MM-SS.csv
```

```
===== */
```

```
bool exportToCSV() {
```

```
    time_t now = time(0);
```

```
    tm *ltm = localtime(&now);
```

```
    char filename[50];
```

```
    strftime(filename, sizeof(filename),
```

```
        "Att-%Y-%m-%d-%H-%M-%S.csv", ltm);
```

```
    ofstream file(filename);
```

```
    if (!file.is_open()) {
```

```
        cout << "? Error: Could not create CSV file.\n";
```

```
        return false;
```

```
    }
```

```
    file << "Name,Attendance\n";
```

```
    for (int i = 0; i < students.size(); i++) {
```

```
        file << students[i] << ", "
```

```
        << (attendanceRecord[i] ? "Present" : "Absent") << "\n";
```

```
    }
```

```

        file.close();

        cout << "? CSV exported successfully: " << filename << endl;

        return true;
    }
};

/* =====

REGRESSION TESTS (Updated for NEW System)

===== */

bool test_addStudent() {
    AttendanceSystem sys;

    sys.addStudent("Ali");

    return sys.getStudents().size() == 1;
}

// ? Test: Attendance should be LOCKED immediately
bool test_attendanceLockedInitially() {
    AttendanceSystem sys;

    sys.addStudent("Ali");

    // First attempt should be locked

    bool allowed = sys.canMarkAttendance();

    return allowed == false; // ? Must return false
}

// ? Test: Attendance should UNLOCK after 2 minutes
bool test_attendanceUnlockAfterTime() {
    AttendanceSystem sys;

    sys.addStudent("Ali");

```

```

// First attempt starts the timer
sys.canMarkAttendance());

// Simulate waiting (for testing: manually reduce the time)
cout << "\n? FOR TESTING: Please wait 2 minutes now...\n";
time_t start = time(0);
while (difftime(time(0), start) < 120) {
    // wait loop
}

return sys.canMarkAttendance() == true; // ? Must unlock now
}

// ? Test: CSV Export should create a NEW timestamp file
bool test_exportToCSV() {
    AttendanceSystem sys;
    sys.addStudent("Ali");
    sys.markPresent(0);
    return sys.exportToCSV(); // ? No filename required now
}

/* =====

MAIN MENU

===== */

int main() {
    /* =====

REGRESSION TESTS

===== */

```

```

    /*cout << "\n===== RUNNING REGRESSION TESTS =====\n";
    cout << "Test Add Student: "
    << (test_addStudent() ? "? PASS" : "? FAIL") << endl;
    cout << "Test Attendance Locked Initially: "
    << (test_attendanceLockedInitially() ? "? PASS" : "? FAIL") << endl;
    cout << "Test Attendance Unlock After 2 Minutes: "
    << (test_attendanceUnlockAfterTime() ? "? PASS" : "? FAIL") << endl;
    cout << "Test CSV Export: "
    << (test_exportToCSV() ? "? PASS" : "? FAIL") << endl;
    cout << "===== \n\n"; */

```

```

AttendanceSystem system;

```

```

int choice;

```

```

cout << "===== \n";

```

```

cout << "  Attendance Management System  \n";

```

```

cout << "===== \n";

```

```

cout << "\n? NOTE: Attendance can be marked AFTER 2 minutes only!\n";

```

```

do {

```

```

    cout << "\nChoose an option:\n";

```

```

    cout << "1. Add Student\n";

```

```

    cout << "2. Mark Attendance\n";

```

```

    cout << "3. Show Attendance\n";

```

```

    cout << "4. Export CSV\n";

```

```

    cout << "0. Exit\n";

```

```

    cout << "Enter choice: ";

```

```

    cin >> choice;

```

```

if (choice == 1) {
    string name;
    cout << "Enter student name: ";
    cin >> name;
    system.addStudent(name);
}

else if (choice == 2) {

    if (!system.canMarkAttendance()) {
        cout << "? Attendance marking is locked for 2 minutes!\n";
        cout << "? Please wait...\n";
        continue;
    }

    vector<string> stds = system.getStudents();

    if (stds.empty()) {
        cout << "No students available to mark.\n";
        continue;
    }

    cout << "\nMark attendance (Enter P for present, anything else = Absent):\n";
    cout << "-----\n";

    for (int i = 0; i < stds.size(); i++) {
        cout << stds[i] << ": ";
        string att;
        cin >> att;
    }
}

```

```
        if (att == "P" || att == "p")
            system.markPresent(i);
    }
    cout << "? Attendance marked!\n";
}

else if (choice == 3) {
    system.showAttendance();
}

else if (choice == 4) {
    system.exportToCSV();
}

} while (choice != 0);

cout << "Program ended.\n";
return 0;
}
```